4. עיבוד תמונה והכנת היעד

כל האלגוריתמים בפרק נמצאים ב־`src/layerforge_l1/fullres_flatten.py` (113 שורות) ונקראים רק מקטעי קוד חד־פעמיים (פרק 3). המטרה: להפוך JPEG עם גרדיאנטים ורעש למפת תוויות (label map) שבה לכל פיקסל יש אחד מ־P צבעים, **בלי להזיז קצוות של קווים ובלי לעבות אותם**. זו הייתה הדרישה המפורשת שלך אחרי שה־flattening הישן הפך קווים אחידים ללא־אחידים.

4.1 FlattenConfig: הפרמטרים

```
src/layerforge_l1/fullres_flatten.py:25-31
  25  @dataclass(frozen=True)
  26  class FlattenConfig:
  27      bilateral_diameter: int = 9
  28      bilateral_sigma_color: float = 24.0
  29      bilateral_sigma_space: float = 5.0
  30      majority_window: int = 5
  31      min_fragment_pixels: int = 40  # ~0.4 mm2 at 0.098 mm/px
```

הערה בקוד: 40 פיקסלים ≈ 0.4 מ"מ² ב־0.098 מ"מ/פיקסל. ההערה נכונה לרשת היעד (2048 פיקסלים = 200 מ"מ). בשלב 1, שבו הפלטה חושבה על התמונה המקורית, אותו ערך מתאים לשטח פיזי מעט שונה, כי שם המיפוי לא שימש לייצור.

4.2 extract_palette: פלטה אוטומטית

`src/layerforge_l1/fullres_flatten.py:76`. **מטרה:** למצוא את הצבעים "האמיתיים" של האיור בלי שגווני מעבר (anti-aliasing) בין שני צבעים ייהפכו לצבע נפרד. בלי הסינון הזה היו נוספים צבעי ביניים, כמו ירוק־שחור לאורך כל קו מתאר, וכל אחד מהם היה דורש לוח נוסף.

```
src/layerforge_l1/fullres_flatten.py:91-113
  91      smooth = cv2.bilateralFilter(
  92          rgb, config.bilateral_diameter, config.bilateral_sigma_color, config.bilateral_sigma_space
  93      )
  94      lab = cv2.cvtColor(smooth, cv2.COLOR_RGB2LAB).astype(np.float32)
  95      gray = cv2.cvtColor(smooth, cv2.COLOR_RGB2GRAY).astype(np.float32)
  96      gradient = np.hypot(cv2.Sobel(gray, cv2.CV_32F, 1, 0), cv2.Sobel(gray, cv2.CV_32F, 0, 1))
  97      interior = cv2.erode((gradient < 40).astype(np.uint8), np.ones((5, 5), np.uint8)).astype(bool)
  98      samples = lab[interior][:: max(1, interior.sum() // 200_000)]
  99      criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 50, 0.5)
 100      _, assignment, centers = cv2.kmeans(samples, max_colors, None, criteria, 4, cv2.KMEANS_PP_CENTERS)
 101      shares = np.bincount(assignment.ravel(), minlength=max_colors) / len(samples)
 102      clusters = [(centers[i], shares[i]) for i in np.argsort(-shares) if shares[i] >= min_share]
 103      merged: list[tuple[np.ndarray, float]] = []
 104      for center, share in clusters:
 105          for index, (kept, kept_share) in enumerate(merged):
 106              if np.linalg.norm(center - kept) < merge_delta:
 107                  total = kept_share + share
 108                  merged[index] = ((kept * kept_share + center * share) / total, total)
 109                  break
 110          else:
 111              merged.append((center, share))
 112      lab_centers = np.clip(np.array([c for c, _ in merged]), 0, 255).astype(np.uint8)
 113      return cv2.cvtColor(lab_centers[None], cv2.COLOR_LAB2RGB)[0]
```

שלב א: החלקה משמרת קצוות (bilateral)

THEORETICAL BACKGROUND הגדרת `cv2.bilateralFilter` עם $d = 9$, $\sigma_c = 24$, $\sigma_s = 5$:

$$\Upsilon(p) = \frac{\sum_{q \in N_9(p)} w_s(p,q) w_c(p,q) I(q)}{\sum_{q \in N_9(p)} w_s(p,q) w_c(p,q)}, \quad w_s = e^{-\frac{\|p-q\|^2}{2\sigma_s^2}}, \quad w_c = e^{-\frac{\delta(I(p),I(q))^2}{2\sigma_c^2}}$$

$N_9(p)$ היא סביבה בקוטר 9 פיקסלים. δ הוא מרחק הצבע. ב־OpenCV, בתמונה בת 3 ערוצים, זהו סכום ההפרשים המוחלטים בין הערוצים (יחידות 0–255). שכן שצבעו שונה מאוד (למשל שחור ליד הצהוב `#EAC007`, עם $\delta = 234 + 191 + 7 = 432$) מקבל משקל $e^{-432^2/(2 \cdot 24^2)} \approx 0$, ולכן הקצה לא מטושטש. רעש JPEG קטן ($\delta \approx 10$) מקבל משקל $e^{-100/1152} \approx 0.92$ ומתמצע.

שלב ב: מעבר ל־Lab וזיהוי פיקסלי פנים

IMPLEMENTED MATHEMATICS שורות 94–97:

$$G(p) = \sqrt{(S_x * g)(p)^2 + (S_y * g)(p)^2}, \qquad \text{interior} = \left[G < 40\right] \ominus B_{5\times 5}$$

g היא תמונת האפור של התמונה המוחלקת. S_x, S_y הם גרעיני Sobel בגודל 3×3, והפלט ב־float32. $\ominus B_{5\times 5}$ היא שחיקה (erosion) בריבוע 5×5: פיקסל נחשב "פנימי" רק אם בכל ריבוע ה־5×5 סביבו הגרדיאנט נמוך מ־40. כך מוסרים גם פיקסלים שנמצאים עד 2 פיקסלים מקצה.

שלב ג: דגימה ו־k-means

IMPLEMENTED MATHEMATICS

$$\text{stride} = \max\left(1, \left\lfloor \frac{N_{\text{int}}}{200000} \right\rfloor\right), \qquad \min_{\mu_1 \dots \mu_{12}} \sum_{x \in X} \min_j \|x - \mu_j\|_2^2$$

X היא קבוצת דגימות Lab בגודל כ־200 אלף. `cv2.kmeans` רץ עם K=12, אתחול k-means++, 4 ניסיונות (נשמר הניסיון עם הקומפקטיות הטובה ביותר) ותנאי עצירה של 50 איטרציות או תזוזה קטנה מ־0.5.

דוגמה אמיתית OBSERVED : בתמונה המקורית 4,209,408 פיקסלים. מתוכם $N_{\text{int}} = 3{,}888{,}812$ פנימיים, ולכן stride = 19 ונלקחו 204,675 דגימות.

שלב ד: סינון ומיזוג אשכולות

IMPLEMENTED MATHEMATICS השורות 101–111. נתח (share) של אשכול: $w_j = |\{x : a(x) = j\}|/|X|$. אשכולות עם $w_j < 0.001$ נזרקים. השאר נסרקים בסדר יורד של w, וכל אחד מתמזג לאשכול הראשון שכבר נשמר ושקרוב אליו מספיק:

$$\|\mu - \kappa\|_2 < 14 \;\Rightarrow\; \kappa \leftarrow \frac{w_\kappa \kappa + w_\mu \mu}{w_\kappa + w_\mu}, \quad w_\kappa \leftarrow w_\kappa + w_\mu$$

זהו ממוצע משוקלל. התוצאה מומרת חזרה ל־RGB. ב־van_gogh_p התקבלו 10 צבעים.

DERIVED המרחק נמדד ביחידות Lab של OpenCV ל־8 ביט: $L_8 = 2.55L^*$,$a_8 = a^* + 128$,$b_8 = b^* + 128$. לכן סף של 14 יחידות שקול ל־$\Delta L^* \approx 5.5$ בציר הבהירות, אבל ל־14 יחידות שלמות בצירי הצבע. ציר הבהירות "מתוח" פי 2.55.

LIMITATION `cv2.kmeans` נקרא בלי לקבע seed, ולכן הרצה חוזרת עשויה להחזיר פלטה מעט שונה. Option B משתמש בפלטה ששמורה ב־`palette.npy`, ולכן הוא ניתן לשחזור מהקובץ, אבל לא מחישוב מחדש.

4.3 snap_to_palette: מיפוי כל פיקסל לצבע בפלטה

`src/layerforge_l1/fullres_flatten.py:34`

```python
src/layerforge_l1/fullres_flatten.py:34-43
  34  def snap_to_palette(rgb: np.ndarray, palette: np.ndarray, config: FlattenConfig = FlattenConfig()) ->
np.ndarray:
  35      smooth = cv2.bilateralFilter(
  36          rgb, config.bilateral_diameter, config.bilateral_sigma_color, config.bilateral_sigma_space
  37      )
  38      lab = cv2.cvtColor(smooth, cv2.COLOR_RGB2LAB).astype(np.float32)
  39      palette_lab = cv2.cvtColor(palette[None].astype(np.uint8), cv2.COLOR_RGB2LAB)[0].astype(np.float32)
  40      distance = ((lab[:, :, None, :] - palette_lab[None, None]) ** 2).sum(-1)
  41      labels = distance.argmin(-1).astype(np.uint8)
  42      labels = majority_filter(labels, len(palette), config.majority_window)
  43      return merge_small_fragments(labels, config.min_fragment_pixels)
```

IMPLEMENTED MATHEMATICS שורות 40–41:

$$\ell(p) = \arg\min_{i \in \{0..P-1\}} \| \mathrm{Lab}(\Upsilon(p)) - \mathrm{Lab}(c_i) \|_2^2$$

דוגמה מספרית עם ערכי הפלטה האמיתיים (Lab של OpenCV): השחור `#000100` הוא (1, 126, 129), ופס הכובע `#354043` הוא (67, 124, 125). לפיקסל מוחלק (60, 124, 126):

$$d^2_{\text{black}} = 59^2 + 2^2 + 3^2 = 3494, \qquad d^2_{\text{band}} = 7^2 + 0^2 + 1^2 = 50 \;\Rightarrow\; \ell = \text{band}$$

זה גם מסביר למה פס הכובע נשאר צבע נפרד: המרחק שלו מהשחור הוא $\sqrt{66^2 + 2^2 + 4^2} \approx 66.2 \gg 14$, ולכן שלב המיזוג לא איחד אותם. Option B איחד אותם ידנית.

DERIVED **זיכרון:** המערך `distance` הוא בגודל $H \times W \times P \times 3$ בפורמט float32. ל־2048 × 1676 × 10 צבעים זה כ־412 מגה־בייט ביניים. זו אחת הסיבות שהרצות על המחשב הזה נעצרו לפעמים בגלל זיכרון.

4.4 majority_filter: הצבעת רוב

`src/layerforge_l1/fullres_flatten.py:46`

```
src/layerforge_l1/fullres_flatten.py:46-50
46  def majority_filter(labels: np.ndarray, count: int, window: int) -> np.ndarray:
47      votes = np.stack(
48          [cv2.boxFilter((labels == i).astype(np.float32), -1, (window, window)) for i in range(count)]
49      )
50      return votes.argmax(0).astype(np.uint8)
```

IMPLEMENTED MATHEMATICS

$$v_i(p) = \frac{1}{25} \sum_{q \in W_5(p)} [\ell(q) = i], \qquad \ell'(p) = \operatorname{argmax}_i v_i(p)$$

`cv2.boxFilter` מנרמל כברירת מחדל, ולכן v_i הוא שבר הפיקסלים מצבע i בחלון 5×5. בתיקו `argmax` מחזיר את האינדקס הנמוך. בשולי התמונה `boxFilter` משתמש בשיקוף (BORDER_REFLECT_101).

DERIVED **למה זה לא מזיז קצה ישר:** בקצה ישר בין שני צבעים, לפיקסל שנמצא צעד אחד בתוך צבע A יש 15 שכנים מ־A ו־10 מ־B בחלון 5×5. A מנצח, והקצה נשאר במקומו. מנגד, "חוד" דק מפיקסל אחד או שניים בולט פחות מ־13 קולות ונמחק. לכן גם קו ברוחב 2 פיקסלים בלבד (0.2 מ"מ) שעובר בתוך שטח בצבע אחד יימחק (10 קולות מול 15). קווי המתאר באיור הזה רחבים בהרבה.

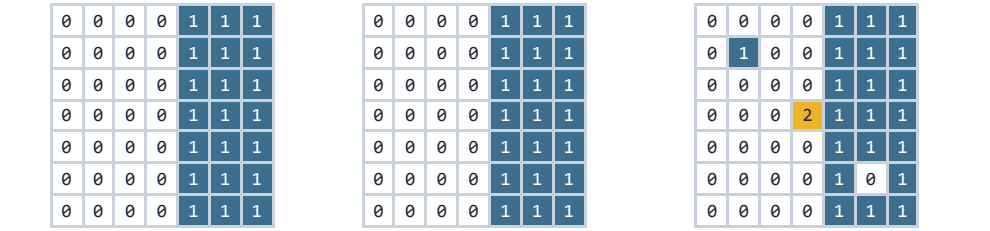
4.5 merge_small_fragments: מיזוג שברים קטנים

`src/layerforge_l1/fullres_flatten.py:53`

```
src/layerforge_l1/fullres_flatten.py:53-73
53  def merge_small_fragments(labels: np.ndarray, min_pixels: int) -> np.ndarray:
54      """Relabel each 4-connected fragment below ``min_pixels`` to its dominant neighbor."""
55
56      labels = labels.copy()
57      for color in np.unique(labels):
58          components, count = ndimage.label(labels == color)
59          if not count:
60              continue
61          areas = np.bincount(components.ravel())
62          for index, region in enumerate(ndimage.find_objects(components), start=1):
63              if areas[index] >= min_pixels:
64                  continue
65              y0, y1 = max(region[0].start - 1, 0), region[0].stop + 1
66              x0, x1 = max(region[1].start - 1, 0), region[1].stop + 1
67              window = labels[y0:y1, x0:x1]
68              mask = components[y0:y1, x0:x1] == index
69              ring = ndimage.binary_dilation(mask) & ~mask
70              neighbors = window[ring]
71              if neighbors.size:
72                  window[mask] = np.bincount(neighbors).argmax()
73      return labels
```

לכל צבע, `ndimage.label` (קשירות 4, ברירת המחדל של scipy) מוצא רכיבים. לכל רכיב F עם $|F| < 40$ מחושבת "טבעת" $\partial F = (F \oplus C) \setminus F$, כאשר C הוא צלב 3×3, וכל F מקבל את הצבע הנפוץ ביותר בטבעת (`bincount().argmax()`). העדכון נעשה במקום, ולכן מיזוג מוקדם משפיע על הטבעות של שברים שנבדקים אחריו.

דוגמה A1: שני המסננים על מערך 7×7 (הופק מהפונקציות האמיתיות)



קלט: 0, 1 ו־2 (רעש: (1,1), (3,3) ו־(5,5))

`majority_filter(w=5)`

`merge_small_fragments(min=2)`

שני המסננים מסירים כאן את שלושת הפיקסלים הבודדים, והקצה האנכי בין 0 ל־1 (בין עמודות 3 ו־4) לא זז. הערכים נוצרו בהרצה של `docs/technical_optB/compute_examples.py`.

4.6 הקטנה ומסגור AD HOC

IMPLEMENTED MATHEMATICS (בקטע החד־פעמי)

$$w = \mathrm{round}\left(1856 \cdot \frac{2048}{2268}\right) = 1676, \qquad x_0 = \left\lfloor \frac{2048 - 1676}{2} \right\rfloor = 186$$

`INTER_AREA` מחשב לכל פיקסל יעד ממוצע משוקלל לפי שטח, ביחס הקטנה 0.903. הפסים $[0, 186)$ ו־$[1862, 2048)$ נצבעים ב־`#FF00FF`. בסך הכול $2 \cdot 186 \cdot 2048 = 761{,}856$ פיקסלים, וזה בדיוק מה שנספר בקובץ.

4.7 תיקוני Option A ו־B: מתמטיקה

IMPLEMENTED MATHEMATICS (בקטע החד־פעמי)

$$T = Y \cap (Y \ominus E_{17}^{-}) \oplus E_{17} \cap W, \qquad \mathrm{opt}_A = \text{target with } T \to \text{black}$$

Y היא מסכת הצהוב `#EAC007`. E_{17} היא אליפסה בקוטר 17 פיקסלים (1.66 מ"מ). $(Y \ominus E) \oplus E$ היא פתיחה (opening) מורפולוגית: כל החלקים של Y שאפשר "לצבוע" באמצעות דיסקה של 1.66 מ"מ שנמצאת כולה בתוך Y. T הוא מה שנשאר מחוץ לפתיחה, כלומר הקוצים הדקים. W הוא חלון סביב רכיב הלחי האדום השמאלי, עם שוליים של 60 פיקסלים. ב־Option B כל פיקסל `#354043` הפך ל־`#000100`.

4.8 exact_index: מתמונה לתוויות

`src/layerforge_l1/experimental_global_planner.py:73`

```python
# src/layerforge_l1/experimental_global_planner.py:73-77
def exact_index(rgb: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Losslessly map exact RGB triplets to deterministic palette indices."""

    palette, inverse = np.unique(rgb.reshape(-1, 3), axis=0, return_inverse=True)
    return inverse.reshape(rgb.shape[:2]).astype(np.uint8), palette.astype(np.uint8)
```

זה מיפוי ללא איבוד מידע: כל שלשת RGB ייחודית מקבלת אינדקס, והפלטה ממוינת לקסיקוגרפית. ב־Option B יש 10 ערכים, ואינדקס 9 הוא `#FF00FF`, ולכן `frame_label=9`. שים לב שמספר הצבעים מוגבל ל־256 (uint8).

אינדקס	צבע	פיקסלים	אינדקס	צבע	פיקסלים
0	#000100	447,051	5	#97CCE5	791,533
1	#3D7033	285,450	6	#BF3934	46,521
2	#609337	88,193	7	#EAC007	1,179,526
3	#7CAE49	138,296	8	#FCFEFE	153,757
4	#8AA1A2	302,121	9	#FF00FF (frame)	761,856

OBSERVED נספר על `flat_2048_framed_optB.png`. היעד מכיל 35 אזורים 4־קשירים, כולל שני פסי המסגרת.